Curs 5 - Algoritmi de Sortare

2024

Cuprins

1 Obiective **1**

2 Limită inferioară pentru un algoritm de sortare prin comparare **1**

3 Analiza complexității medii - InsertionSort, QuickSort **2**

3.1 InsertionSort 2

3.2 QuickSort 3

3.2.1 Metoda 1 4

3.2.2 Metoda 2 4

4 Construirea unui heap binar în timp liniar **6**

1 Obiective

În acest document ne propunem să parcurgem câteva demonstrații interesante discutate la curs. Pentru a simplifica analiza, în acest document vom considera că:

- Secvența primită la intrare este o permutare aleatoare a N chei **distincte**.
- Fiecare permutare are **o probabilitate egală** de a fi primită la intrare.
- Complexitatea algoritmilor de sortare prin comparare este determinată de **numărul de operații de comparare** realizate pe parcursul execuției.

2 Limită inferioară pentru un algoritm de sortare prin comparare

Teoremă 1:

Complexitatea oricărui algoritm de sortare prin comparare este $\Omega(NlogN)$.

Putem modela execuția unui algoritm de sortare prin comparare cu ajutorul unui arbore de decizie (v. Figura 1). Fiecare nod intern din arbore descrie o operație de comparare între două elemente iar fiecare frunză ilustrează o posibilă soluție a procesului de sortare.

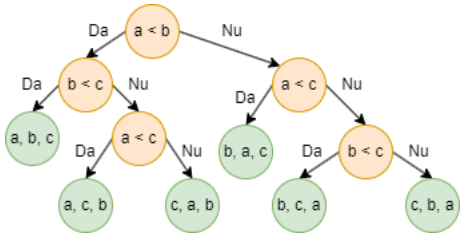


Figura 1: Arbore de decizie care descrie o posibilă secvență de comparații necesare pentru a sorta trei chei distincte.

Numărul maxim de arce parcurse, pornind de la rădăcină până la orice frunză, corespunde cu numărul maxim de decizii luate de algoritm pentru stabilirea soluției. Ne propunem să investigăm care este înățimea minimă (h) a arborelui asociat de decizie necesară pentru a sorta corect orice secvență alcătuită din N elemente distincte.

O primă observație pe care o putem face este că numărul de frunze va fi **cel puțin** $N!$ pentru a acoperi toate cazurile posibile (numărul total de permutări). Pe de altă parte, un arbore binar de înălțime maximă h are **cel mult** 2^h **frunze**:

$$N! \leq \text{numărul de frunze} \leq 2^h$$
$$log_2(N!) \leq h$$

Pentru a încadra asimptotic $log_2(N!)$ putem să identificăm o limită inferioară, respectiv superioară:

$$log_2(N!) = log_2(1) + log_2(2) + \cdots + \log_2(\frac{N}{2}) + \cdots + log_2(N-1) + log_2(N)$$
$$log_2(N!) \geq \frac{N}{2} log_2(\frac{N}{2}) \in \theta(NlogN)$$
$$log_2(N!) \leq Nlog_2(N) \in \theta(NlogN)$$
$$log_2(N!) \in \theta(NlogN)$$

Prin urmare, un algoritm de sortare prin comparare necesită $\Omega(NlogN)$ operații de comparare pentru a putea distinge între cele $N!$ posibile soluții.

3 Analiza complexității medii - InsertionSort, QuickSort

Definiție 1: Complexitate medie a unui algoritm

- Fie I mulțimea tuturor intrărilor posibile pentru algoritmul pe care vrem să îl studiem.
- Fie $X(i)$ o variabilă aleatoare care exprimă numărul de operații elementare (complexitatea), realizate de algoritm pentru intrarea $i \in I$.
- Fie $p(i)$ probabilitatea de a întâlni intrarea i.

Atunci, complexitatea medie a algoritmului este:

$$E[X] = \sum_{i \in I} p(i) \cdot X(i) \tag{1}$$

Ne propunem să calculăm complexitatea pe cazul mediu pentru doi algoritmi studiați: InsertionSort, respectiv QuickSort.

3.1 InsertionSort

```
1:  function InsertionSort(A[1..n])
2:      for i = 2; i ≤ n; i++ do
3:          j = i − 1
4:          while j > 0 and A[j + 1] < A[j] do
5:              swap(A[j], A[j + 1])
6:              j− = 1
7:          end while
8:          i+ = 1
9:      end for
10: end function
```

Primul for se execută întotdeauna de n ori. Dacă secvența primită la intrare este deja sortată crescător, execuția programului nu intră deloc în while, iar dacă secvența este sortată descrescător vom verifica condiția din while de i ori.

Astfel, în cel mai avantajos caz, complexitatea va fi $O(n)$, iar în cel mai dezavantajos caz:

$$T(n) = ct \cdot \sum_{i=2}^{n} i \in O(n^2)$$

Follow-up 1

Ce complexitate obținem dacă, pentru fiecare i, s-ar executa întotdeauna doar $\frac{1}{10}$ din pașii corespunzători cazului cel mai defavorabil?

Cum se comportă însă algoritmul pe cazul mediu? De exemplu, pentru al doilea element din șir avem 2 posibilități: elementul rămâne pe poziția sa inițială sau se interschimbă cu primul element. În prima situație, condiția de pe linia 4 se va executa o singură dată, respectiv de două ori dacă elementul va fi mutat pe prima poziție. Deoarece primim la intrare o permutare aleatoare, vom considera că ambele posibilități au o probabilitate egală, $\frac{1}{2}$.

Mai general, costul mediu pentru mutarea elementului i este:

$$\sum_{j=1}^{i} \frac{1}{i} \cdot j$$

Prin urmare, complexitatea medie pentru întregul algoritm este:

$$T(n) = ct \cdot \sum_{i=2}^{n} \sum_{j=1}^{i} \frac{j}{i} \qquad \text{(folosim identitatea } \sum_{i=1}^{n} i = \frac{n(n+1)}{2}\text{)}$$

$$T(n) = ct \cdot \sum_{i=2}^{n} \frac{1}{i} \cdot \frac{i(i+1)}{2}$$

$$T(n) = ct \cdot \sum_{i=2}^{n} \frac{(i+1)}{2}$$

$$T(n) \in \theta(n^2)$$

3.2 QuickSort

```
1:  function PARTITION(A[1..n], start, end)
2:      pivot = A[end]
3:      i = start − 1
4:
5:      for j = start to end − 1 do
6:          if A[j] < pivot then
7:              i = i + 1
8:              swap(A[i], A[j])
9:          end if
10:     end for
11:
12:     swap(A[i + 1], A[end])
13:     return i + 1
14: end function

15: function QUICKSORT(A[1..n], start, end)
16:     if start ≥ end then return
17:     end if
18:     p = Partition(A, start, end) // Divide - θ(N)
19:     QuickSort(A, start, p − 1)   // Impera
20:     QuickSort(A, p + 1, end)     // Impera
21: end function
```

La curs am remarcat că, pentru cazul cel mai avantajos, funcția de partiționare alege întotdeauna drept pivot elementul median, ceea ce determină o spargere echilibrată în două subprobleme de dimensiune egală.

$$T_{Best}(n) = 2 \cdot T_{Best}(\frac{n}{2}) + \theta(n)$$
$$T_{Best}(n) \in O(nlogn)$$

Pentru cazul cel mai dezavantajos însă, elementul ales este chiar cel mai mic sau cel mai mare din secvență, ceea ce conduce la următoarea relație de recurență:

$$T_{Worst}(n) = T_{Worst}(n-1) + T_{Worst}(1) + \theta(n)$$
$$T_{Worst}(n) \in O(n^2)$$

Follow-up 2

Ce complexitate asimptotică obținem dacă subproblema mai mică are întotdeauna, începând cu o anumită valoare n_0 ..:

- 10% din dimensiunea problemei inițiale ?
- K elemente, unde $K < n_0$ este o constantă?

3.2.1 Metoda 1

Deoarece considerăm că primim la intrare o permutare aleatoare, vom considera că avem o probabilitate egală de a alege drept pivot oricare dintre elementele de la 1 la n. Astfel, avem o probabilitate de $\frac{1}{n}$ de a alege drept pivot elementul de pe poziția i din șirul sortat. În acest caz, cele două subprobleme care rămân de rezolvat vor avea dimensiunea $i-1$, respectiv $n-i$. Prin urmare, putem caracteriza complexitatea pe cazul mediu în felul următor:

$$T(n) = \begin{cases} 0 & \text{if } n < 2, \\ \frac{1}{n}\sum_{i=1}^{n-1}(T(i-1) + T(n-i)) + ct \cdot (n-1) & \text{altfel.} \end{cases}$$

Rafinăm succesiv expresia pentru cazul general:

$$T(n) = \frac{1}{n}\sum_{i=1}^{n-1}(T(i-1) + T(n-i)) + ct \cdot (n-1) \quad \text{(se repetă de două ori suma } \sum_{i=1}^{n-1} T(i))$$
$$T(n) = \frac{2}{n}\sum_{i=1}^{n-1} T(i) + ct \cdot (n-1) \quad \text{(înmulțim cu n)}$$
$$n \cdot T(n) = 2\sum_{i=1}^{n-1} T(i) + ct \cdot n(n-1) \quad \text{(scriem ecuația echivalentă pentru } T(n-1))$$
$$(n-1) \cdot T(n-1) = 2\sum_{i=1}^{n-2} T(i) + ct \cdot (n-1)(n-2) \quad \text{(scădem noua ecuație din cea precedentă)}$$
$$n \cdot T(n) - (n-1) \cdot T(n-1) = 2\ T(n-1) + ct \cdot (n-1)(n-(n-2))$$
$$n \cdot T(n) = (n+1)\ T(n-1) + 2 \cdot ct \cdot (n-1) \quad \text{(împărțim la n)}$$
$$T(n) = \frac{n+1}{n}(T(n-1) + 2 \cdot ct) \quad \text{(iterăm pentru valori succesiv mai mici ale lui n)}$$
$$T(n) = \frac{n+1}{2}T(2) + 2 \cdot ct \cdot (n+1)\sum_{i=2}^{n}\frac{1}{i} \quad \text{(folosim identitatea } \sum_{i=2}^{n}\frac{1}{i} = \theta(log(n))$$
$$T(n) \in O(nlog(n))$$

3.2.2 Metoda 2

Alternativ, vom încerca să calculăm probabilitatea pentru fiecare pereche de elemente din secvența să fie comparate pe parcursul execuției programului.

Fie s_i al i-lea element din șirul sortat. Fie X_{ij} o variabilă aleatoare cu semnficația:

$$X_{ij} = \begin{cases} 1 & \text{elementul } s_i \text{ este comparat cu elementul } s_j \\ 0 & altfel \end{cases}$$

Fie $Comp = \sum_i^{n-1} \sum_{j=i+1}^{n} X_{ij}$ numărul total de comparații realizate de algoritm.

Fie $p(X_{ij} = 1)$ probabilitatea ca cele două elemente să fie comparate. Observăm că aceasta este determinată de ordinea în care alegem pivotul. Costul mediu asociat variabilei X_{ij} este:

$$E[X_{ij}] = 1 \cdot p(X_{ij} = 1) + 0 \cdot p(X_{ij} = 0) = p(X_{ij} = 1) \tag{2}$$

Fie p poziția finală a pivotului ales în funcția de partiționare și două elemente s_i și s_j, cu $i < j$. Sunt trei situații care pot avea loc:

1. $p < s_i$ sau $p > s_j$: Ambele elemente se vor afla fie în dreapta, respectiv în stânga pivotului dupa apelarea funcției de partiționare.
2. $p == s_i$ sau $p == s_j$: Cele două elemente vor fi comparate o singură dată. Elementul care corespunde pivotului nu va mai face parte din apelurile recursive ulterioare.
3. $s_i < p < s_j$: Cele două elemente nu vor fi niciodată comparate, deoarece vor fi trimise separat la primul, respectiv al doilea apel recursiv.

Observăm că doar în situația 2 sau 3 este afectată probabilitatea ca cele două elemente să fie comparate. Concret, cele două chei sunt comparate în doar 2 din j - i + 1 cazuri:

$$p(X_{ij} = 1) = \frac{2}{j - i + 1} \tag{3}$$

Fie C, o variabilă aleatoare care descrie numărul total de comparații realizate de algoritm.

$$C = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

Prin urmare, numărul mediu de comparații, $E[C]$, este:

$$\begin{aligned}
E[C] &= E[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] && \text{(folosim identitatea E[X + Y] = E[X] + E[Y])} \\
&= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} E[X_{ij}] && \text{(v. Eq. 2)} \\
&= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} p(X_{ij} = 1) && \text{(v. Eq. 3)} \\
&= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j - i + 1} && \text{(identificăm o limită superioară)} \\
&\leq \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j} \\
&\leq 2(n-1) \sum_{j=2}^{n} \frac{1}{j} && \text{(folosim identitatea } \sum_{i=2}^{n} \frac{1}{i} = \theta(log(n)) \\
&\leq 2(n-1)ln(n) \\
E[C] &\in O(nlog(n))
\end{aligned}$$

Follow-up 3:

Cum putem îmbunătăți complexitatea spațială a algoritmului QuickSort?

Follow-up 4:

Cum putem îmbunătăți funcția de partiționare dacă permitem existența elementelor duplicate?

4 Construirea unui heap binar în timp liniar

Un (max)-heap binar este un arbore binar care respectă două constrângeri suplimentare:

- *Heap Property*: Cheia asociată fiecărui nod este mai mare sau egală față de cea asociată copiilor săi.
- Fiecare nivel din arbore este complet cu excepția eventual a ultimului nivel, care se completează cu noduri de la stânga la dreapta.

Un array $A[1], A[2], \ldots, A[n]$ poate modela un heap binar dacă pentru $\forall i = 2, \ldots, n$, $A[\lfloor \frac{i}{2} \rfloor] \geq A[i]$. Pe prima poziție, $A[1]$, se află elementul cu prioritatea maximă.

O abordare intuitivă pentru construirea unui heap binar cu n elemente presupune apelarea operației de inserare separat pentru fiecare element din secvență. Deoarece operația $insert()$ are o complexitate de $O(log(n))$, această abordare are o complexitate de $O(nlogn)$.

Putem însă să construim heap-ul mai eficient. Fie o secvența arbitrară de numere salvată în array-ul A. Pentru ca aceasta să reprezinte un heap, vom apela iterativ operația de *downHeap* pe rând, pentru toate elementele care ar corespunde ultimului nivel, penultimului nivel, etc.

```
1: function downHeap(A[1..n], index)
2:     nextIndex = index
3:     if index * 2 < n and A[index * 2] > A[nextIndex] then
4:         nextIndex = index * 2
5:     end if
6:     if index * 2 + 1 ≤ n and A[index * 2 + 1] > A[nextIndex] then
7:         nextIndex = index * 2 + 1
8:     end if
9:     if index ≠ nextIndex then
10:        swap(A[index], A[nextIndex]
11:        downHeap(A, nextIndex)
12:    end if
13: end function

14: function heapify(A[1..n])
15:    for i = n/2; i; i − − do downHeap(A, i);
16:    end for
17: end function
```

Ce complexitate are această abordare? Fie $h = \lfloor n \rfloor$, nivelul maxim din heap. Vom analiza pentru fiecare nivel numărul maxim de operații de apeluri recursive ale operației *downHeap*:
Astfel, formula generală este:

Nivel	Număr de noduri	Număr de apeluri per nod a funcției downHeap (worst-case)
h	$O(\frac{n}{2})$	0
h - 1	$O(\frac{n}{2^2})$	1
h - 2	$O(\frac{n}{2^3})$	2
...	...	...
i	$O(\frac{n}{2^i})$	i - 1
...	...	...
1	1	h - 1

$$
\begin{aligned}
T(n) &= 1 \cdot \frac{n}{2} + 2 \cdot \frac{n}{2^2} + 3 \cdot \frac{n}{2^3} + \cdots + (i-1) \cdot \frac{n}{2^{i+1}} + (h-1) \cdot 1 \\
&= \sum_{i=1}^{(h-1)} i \cdot \frac{n}{2^{i+1}} \\
&= n \sum_{i=1}^{(h-1)} i \cdot \frac{1}{2^{i+1}}
\end{aligned}
$$

Fie $x = \frac{1}{2}, m = (h-1)$. Vom calcula separat $\sum_{i=1}^{m} i \cdot x^{i+1}$:

$$
\begin{aligned}
\sum_{i=1}^{m} i \cdot x^{i+1} &= x^2 \sum_{i=1}^{m} i \cdot x^{i-1} \\
&= x^2 \sum_{i=1}^{m} \frac{d}{dx} x^i \\
&= x^2 \frac{d}{dx} [\sum_{i=1}^{m} x^i] \\
&\leq x^2 \frac{d}{dx} \sum_{i=1}^{\infty} x^i \\
&\leq \frac{1}{x^2} \frac{d}{dx} (\frac{1}{1-x}) \\
&\leq \frac{1}{x^2} \frac{1}{(1-x)^2} && \text{(înlocuim x)} \\
&\leq \frac{1}{4} \frac{1}{(1-\frac{1}{2})^2} = \frac{1}{4} \cdot 4 = 1
\end{aligned}
$$

Prin urmare, $T(n) \in O(n)$.

Follow-up 5:

Cum am putea folosi un heap pentru a calcula eficient top K cele mai mari elemente dintr-o secvență?

Follow-up 6:

Cum am putea modifica algoritmul QuickSort pentru a rezolva aceeași problemă eficient?

Referințe

[1] Thomas H Cormen et al. *Introduction to algorithms*. MIT press, 2022.

[2] Tim Roughgarden. *Algorithms illuminated*. Soundlikeyourself publishing, 2017.

[3] Robert Sedgewick and Kevin Wayne. *Algorithms: Part I*. Addison-Wesley Professional, 2014.